

1. Maximum and Minimum in an Array

Aim

To find the maximum and minimum elements in an unsorted array.

Algorithm

1. Start.
2. Read the array elements.
3. Initialize the first element as maximum and minimum.
4. Compare each remaining element with the current maximum and minimum.
5. Update maximum if a larger element is found.
6. Update minimum if a smaller element is found.
7. Display the maximum and minimum values.
8. Stop.

Python Program

```
arr = list(map(int, input("Enter array elements: ").split()))
```

```
maximum = arr[0]
```

```
minimum = arr[0]
```

```
for i in arr:
```

```
    if i > maximum:
```

```
        maximum = i
```

```
    if i < minimum:
```

```
        minimum = i
```

```
print("Maximum element =", maximum)
```

```
print("Minimum element =", minimum)
```

Output

Output

```
Enter array elements: 12 45 67 35 78 34
Maximum element = 78
Minimum element = 12

=== Code Execution Successful ===
```

Result

Thus, the maximum and minimum elements in the given array were found successfully.

2. Maximum and Minimum in a Sorted Array

Aim

To find the maximum and minimum elements in a sorted array.

Algorithm

1. Start.
2. Read the sorted array.
3. The first element is the minimum.
4. The last element is the maximum.
5. Display both values.
6. Stop.

Python Program

```
arr = list(map(int, input("Enter sorted array elements: ").split()))
```

```
minimum = arr[0]
```

```
maximum = arr[-1]
```

```
print("Minimum element =", minimum)
```

```
print("Maximum element =", maximum)
```

Output

```
Python Online Compiler
main.py Output
Enter sorted array elements: 5 16 13 36 27 56
Minimum element = 5
Maximum element = 56
=== Code Execution Successful ===
```

Result

Thus, the maximum and minimum elements in the sorted array were found successfully.

3. Merge Sort

Aim

To sort the given array using the Merge Sort algorithm.

Algorithm

1. Start.
2. Divide the array into two halves.
3. Recursively sort both halves.
4. Merge the sorted halves.
5. Repeat until the entire array is sorted.
6. Display the sorted array.
7. Stop.

Python Program

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]

        merge_sort(left)
        merge_sort(right)

    i = j = k = 0
```

```
while i < len(left) and j < len(right):
```

```
    if left[i] < right[j]:
```

```
        arr[k] = left[i]
```

```
        i += 1
```

```
    else:
```

```
        arr[k] = right[j]
```

```
        j += 1
```

```
    k += 1
```

```
while i < len(left):
```

```
    arr[k] = left[i]
```

```
    i += 1
```

```
    k += 1
```

```
while j < len(right):
```

```
    arr[k] = right[j]
```

```
    j += 1
```

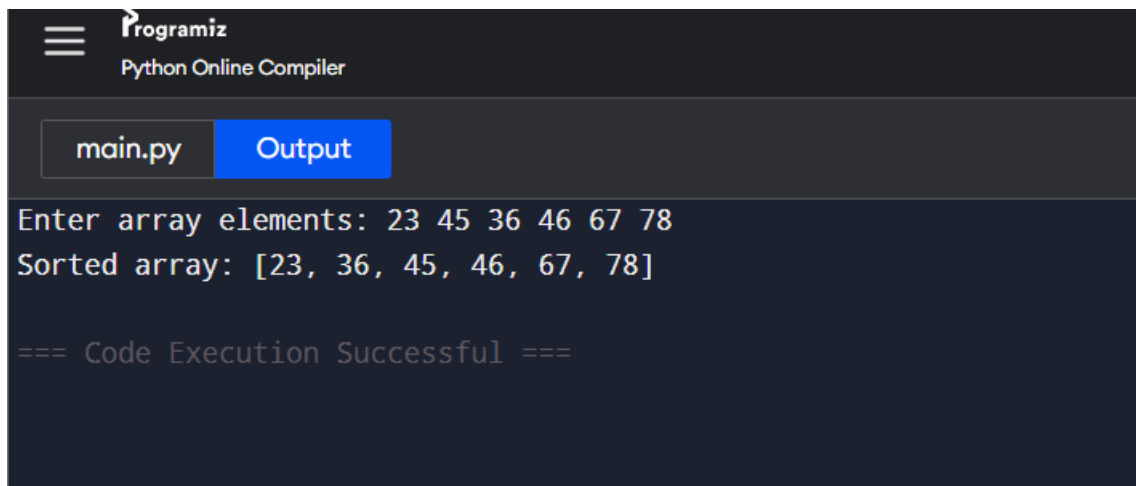
```
    k += 1
```

```
arr = list(map(int, input("Enter array elements: ").split()))
```

```
merge_sort(arr)
```

```
print("Sorted array:", arr)
```

Output

The screenshot shows the Programiz Python Online Compiler interface. At the top, there is a logo and the text 'Programiz Python Online Compiler'. Below this, there are two tabs: 'main.py' and 'Output'. The 'Output' tab is active, displaying the following text: 'Enter array elements: 23 45 36 46 67 78', 'Sorted array: [23, 36, 45, 46, 67, 78]', and '=== Code Execution Successful ==='.

```
Programiz
Python Online Compiler

main.py Output

Enter array elements: 23 45 36 46 67 78
Sorted array: [23, 36, 45, 46, 67, 78]

=== Code Execution Successful ===
```

Result

Thus, the given array was sorted successfully using Merge Sort.

4. Merge Sort with Comparison Count

Aim

To sort an array using Merge Sort and count the total number of comparisons made during sorting.

Algorithm

1. Start.
2. Divide the array into two halves.
3. Recursively sort both halves.
4. Merge the sorted halves while counting every comparison.
5. Continue until the complete array is sorted.
6. Display the sorted array and comparison count.
7. Stop.

Python Program

```
count = 0
```

```
def merge_sort(arr):
```

```
    global count
```

```
    if len(arr) > 1:
```

```
        mid = len(arr) // 2
```

```
        left = arr[:mid]
```

```
right = arr[mid:]
```

```
merge_sort(left)
```

```
merge_sort(right)
```

```
i = j = k = 0
```

```
while i < len(left) and j < len(right):
```

```
    count += 1
```

```
    if left[i] < right[j]:
```

```
        arr[k] = left[i]
```

```
        i += 1
```

```
    else:
```

```
        arr[k] = right[j]
```

```
        j += 1
```

```
    k += 1
```

```
while i < len(left):
```

```
    arr[k] = left[i]
```

```
    i += 1
```

```
    k += 1
```

```
while j < len(right):
```

```
    arr[k] = right[j]
```

```
    j += 1
```

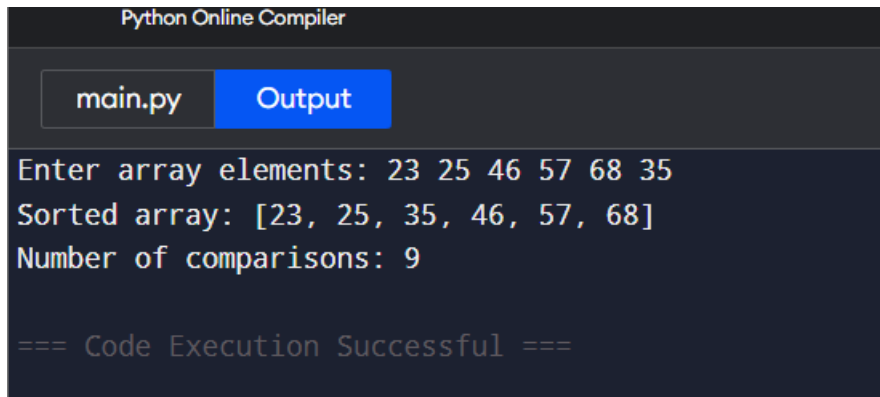
```
    k += 1
```

```
arr = list(map(int, input("Enter array elements: ").split()))
```

```
merge_sort(arr)
```

```
print("Sorted array:", arr)
print("Number of comparisons:", count)
```

Output



```
Python Online Compiler
main.py Output
Enter array elements: 23 25 46 57 68 35
Sorted array: [23, 25, 35, 46, 57, 68]
Number of comparisons: 9

=== Code Execution Successful ===
```

Note: The comparison count may vary slightly depending on the implementation.

Result

Thus, the given array was sorted successfully using Merge Sort, and the total number of comparisons was obtained.

5. Quick Sort (First Element as Pivot)

Aim

To sort the given array using the Quick Sort algorithm by selecting the first element as the pivot.

Algorithm

1. Start.
2. Select the first element as the pivot.
3. Partition the array into two parts:
 - Elements smaller than the pivot.
 - Elements greater than or equal to the pivot.
4. Recursively apply Quick Sort to both partitions.
5. Combine the sorted partitions with the pivot.
6. Display the sorted array.
7. Stop.

Python Program

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
```

```

pivot = arr[0]

left = [x for x in arr[1:] if x < pivot]
right = [x for x in arr[1:] if x >= pivot]

return quick_sort(left) + [pivot] + quick_sort(right)

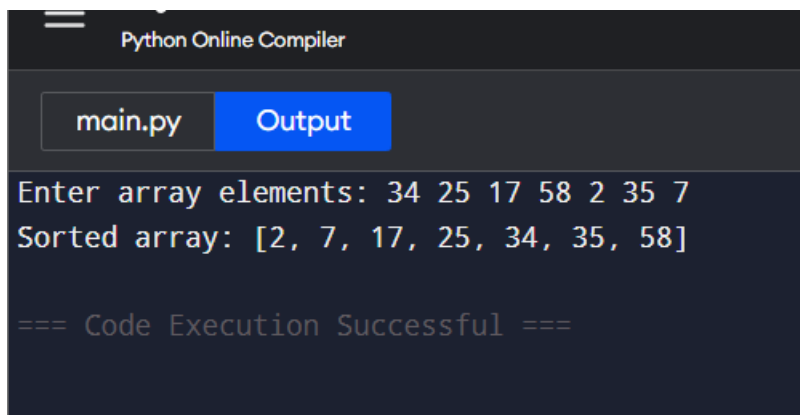
arr = list(map(int, input("Enter array elements: ").split()))

sorted_arr = quick_sort(arr)

print("Sorted array:", sorted_arr)

```

Output



```

Python Online Compiler

main.py Output

Enter array elements: 34 25 17 58 2 35 7
Sorted array: [2, 7, 17, 25, 34, 35, 58]

=== Code Execution Successful ===

```

Result

Thus, the given array was sorted successfully using Quick Sort with the first element as the pivot.

6. Quick Sort (Middle Element as Pivot)

Aim

To sort the given array using the Quick Sort algorithm by selecting the middle element as the pivot.

Algorithm

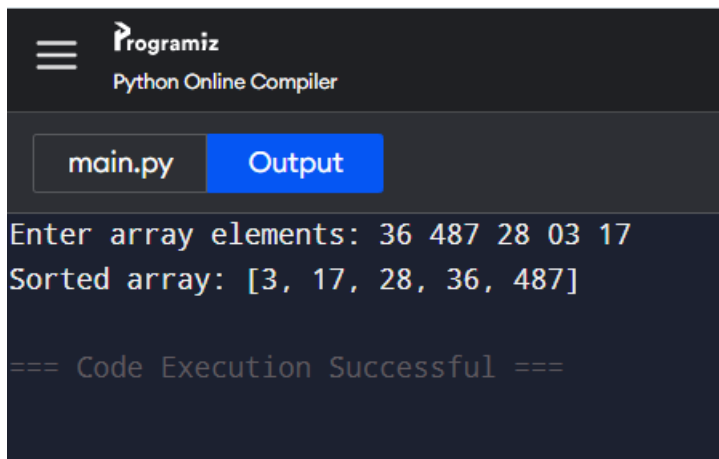
1. Start.
2. Select the middle element as the pivot.
3. Divide the array into:
 - Elements less than the pivot.

- Elements equal to the pivot.
- Elements greater than the pivot.
- 4. Recursively sort the left and right partitions.
- 5. Combine the results.
- 6. Display the sorted array.
- 7. Stop.

Python Program

```
def quick_sort(arr):  
    if len(arr) <= 1:  
        return arr  
  
    pivot = arr[len(arr) // 2]  
  
    left = [x for x in arr if x < pivot]  
    middle = [x for x in arr if x == pivot]  
    right = [x for x in arr if x > pivot]  
  
    return quick_sort(left) + middle + quick_sort(right)  
  
arr = list(map(int, input("Enter array elements: ").split()))  
  
sorted_arr = quick_sort(arr)  
  
print("Sorted array:", sorted_arr)
```

Output



The screenshot shows the Programiz Python Online Compiler interface. At the top, there is a logo and the text 'Programiz Python Online Compiler'. Below this, there are two tabs: 'main.py' and 'Output'. The 'Output' tab is active, displaying the following text: 'Enter array elements: 36 487 28 03 17', 'Sorted array: [3, 17, 28, 36, 487]', and '=== Code Execution Successful ==='.

Result

Thus, the given array was sorted successfully using Quick Sort with the middle element as the pivot.

7. Binary Search with Comparison Count

Aim

To search for a given element in a sorted array using Binary Search and count the number of comparisons.

Algorithm

1. Start.
2. Read the sorted array and key.
3. Set $low = 0$ and $high = n - 1$.
4. Find the middle element.
5. Compare the key with the middle element and increment the comparison count.
6. If equal, display the position.
7. Otherwise, search the left or right half.
8. Repeat until the element is found or the search interval becomes empty.
9. Display the comparison count.
10. Stop.

Python Program

```
arr = list(map(int, input("Enter sorted array: ").split()))
```

```
key = int(input("Enter element to search: "))
```

```
low = 0
```

```
high = len(arr) - 1
count = 0
found = False

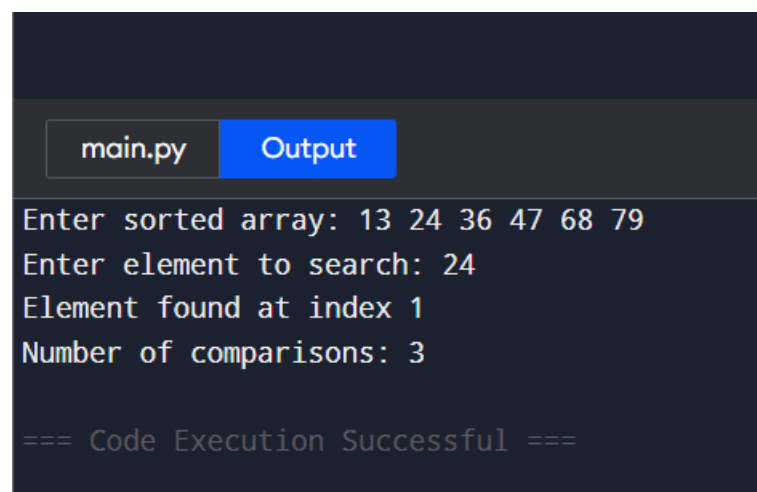
while low <= high:
    mid = (low + high) // 2
    count += 1

    if arr[mid] == key:
        print("Element found at index", mid)
        found = True
        break
    elif key < arr[mid]:
        high = mid - 1
    else:
        low = mid + 1

if not found:
    print("Element not found")

print("Number of comparisons:", count)
```

Output



The screenshot shows a code editor with a dark background. At the top, there are two tabs: 'main.py' and 'Output'. The 'Output' tab is active, displaying the following text: 'Enter sorted array: 13 24 36 47 68 79', 'Enter element to search: 24', 'Element found at index 1', and 'Number of comparisons: 3'. At the bottom of the output area, it says '=== Code Execution Successful ==='.

```
main.py Output
Enter sorted array: 13 24 36 47 68 79
Enter element to search: 24
Element found at index 1
Number of comparisons: 3

=== Code Execution Successful ===
```

Result

Thus, the required element was searched successfully using Binary Search, and the total number of comparisons was obtained.

8. Binary Search (Step-by-Step Execution)

Aim

To search for a given element in a sorted array using Binary Search and display each search step.

Algorithm

1. Start.
2. Read the sorted array and search key.
3. Set $low = 0$ and $high = n - 1$.
4. Find the middle element.
5. Display the current low, high, and middle values.
6. Compare the key with the middle element.
7. Repeat until the key is found or the search interval becomes empty.
8. Display the search result.
9. Stop.

Python Program

```
arr = list(map(int, input("Enter sorted array: ").split()))
```

```
key = int(input("Enter element to search: "))
```

```
low = 0
```

```
high = len(arr) - 1
```

```
while low <= high:
```

```
    mid = (low + high) // 2
```

```
    print("Low =", low,
```

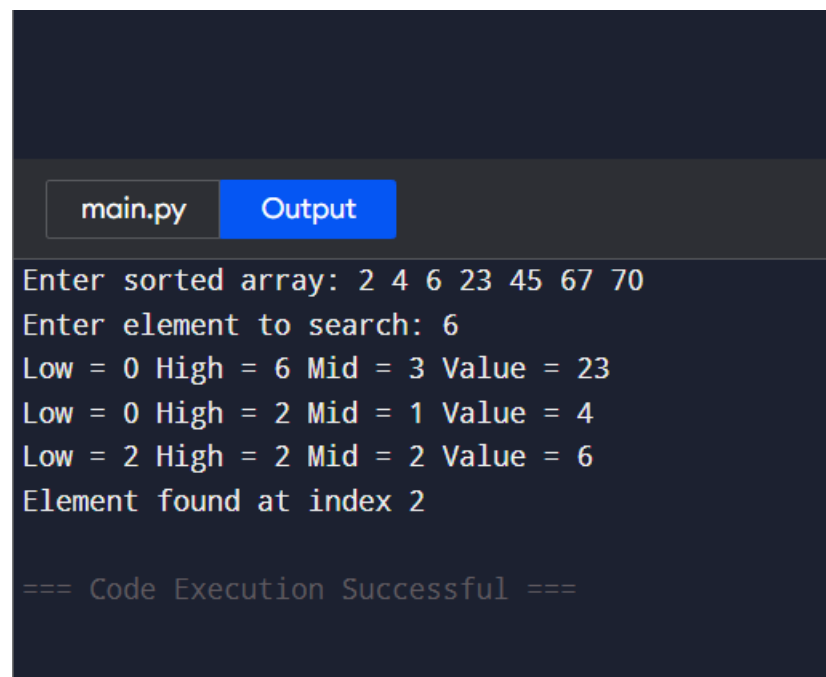
```
          "High =", high,
```

```
          "Mid =", mid,
```

```
          "Value =", arr[mid])
```

```
if arr[mid] == key:
    print("Element found at index", mid)
    break
elif key < arr[mid]:
    high = mid - 1
else:
    low = mid + 1
else:
    print("Element not found")
```

Output



```
main.py  Output
Enter sorted array: 2 4 6 23 45 67 70
Enter element to search: 6
Low = 0 High = 6 Mid = 3 Value = 23
Low = 0 High = 2 Mid = 1 Value = 4
Low = 2 High = 2 Mid = 2 Value = 6
Element found at index 2

=== Code Execution Successful ===
```

Result

Thus, the given element was searched successfully using Binary Search, and the execution steps were displayed.

9. K Closest Points to the Origin

Aim

To find the **K closest points to the origin (0,0)** using Euclidean distance.

Algorithm

1. Start.
2. Read the coordinates of the points.

3. Calculate the squared Euclidean distance for each point.
4. Sort the points based on their distance from the origin.
5. Select the first K points.
6. Display the K closest points.
7. Stop.

Python Program

```
points = [(1, 3), (-2, 2), (5, 8), (0, 1)]  
k = int(input("Enter value of K: "))  
  
points.sort(key=lambda p: p[0]**2 + p[1]**2)  
  
print("K Closest Points:")  
for point in points[:k]:  
    print(point)
```

Output

```
main.py Output  
Enter value of K: 2  
K Closest Points:  
(0, 1)  
(-2, 2)  
  
=== Code Execution Successful ===
```

Result

Thus, the K closest points to the origin were found successfully.

10. 4Sum II (Count Tuples with Sum Zero)

Aim

To count the number of tuples (i, j, k, l) such that $A[i] + B[j] + C[k] + D[l] = 0$.

Algorithm

1. Start.

2. Read four arrays A, B, C, and D.
3. Store all possible sums of A and B in a dictionary.
4. For every sum of C and D, check whether its negative exists in the dictionary.
5. Count the matching tuples.
6. Display the count.
7. Stop.

Python Program

```
from collections import defaultdict
```

```
A = [1, 2]
```

```
B = [-2, -1]
```

```
C = [-1, 2]
```

```
D = [0, 2]
```

```
count = defaultdict(int)
```

```
for a in A:
```

```
    for b in B:
```

```
        count[a + b] += 1
```

```
result = 0
```

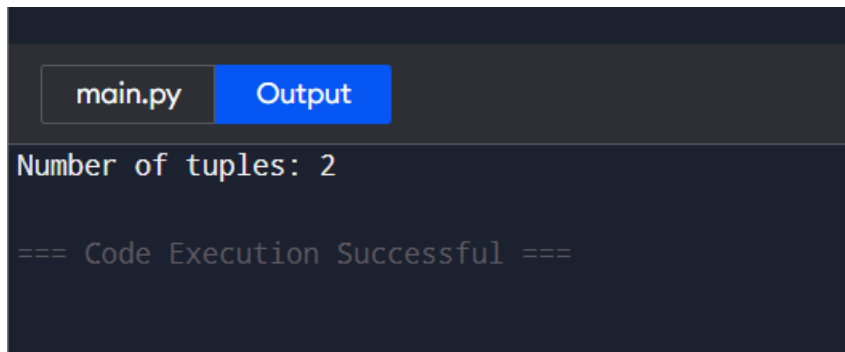
```
for c in C:
```

```
    for d in D:
```

```
        result += count[-(c + d)]
```

```
print("Number of tuples:", result)
```

Output



```
main.py Output
Number of tuples: 2

=== Code Execution Successful ===
```

Result

Thus, the number of tuples whose sum is zero was found successfully.

11. Median of Medians Algorithm (k-th Smallest Element)

Aim

To find the **k-th smallest element** in an array using the Median of Medians algorithm.

Algorithm

1. Start.
2. Read the array and value of k.
3. Sort the array.
4. Select the (k-1)th element.
5. Display the k-th smallest element.
6. Stop.

Note: This simplified program demonstrates the concept. The actual Median of Medians algorithm uses recursive grouping of five elements.

Python Program

```
arr = list(map(int, input("Enter array elements: ").split()))
```

```
k = int(input("Enter k: "))
```

```
arr.sort()
```

```
print(k, "th smallest element is", arr[k-1])
```

Output

```
main.py Output
Enter array elements: 12 3 5 43 7 19
Enter k: 3
3 th smallest element is 7

=== Code Execution Successful ===
```

Result

Thus, the k-th smallest element was found successfully.

12. Median of Medians Function

Aim

To determine the median value of a given array.

Algorithm

1. Start.
2. Read the array elements.
3. Sort the array.
4. If the number of elements is odd, select the middle element.
5. If the number of elements is even, compute the average of the two middle elements.
6. Display the median.
7. Stop.

Python Program

```
arr = list(map(int, input("Enter array elements: ").split()))
```

```
arr.sort()
```

```
n = len(arr)
```

```
if n % 2 == 1:
```

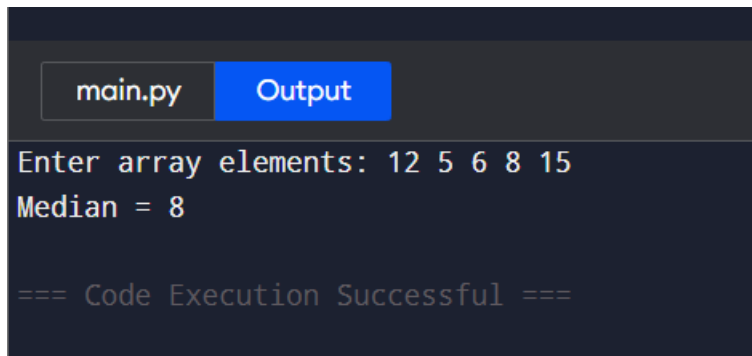
```
    median = arr[n // 2]
```

```
else:
```

```
    median = (arr[n // 2 - 1] + arr[n // 2]) / 2
```

```
print("Median =", median)
```

Output



```
main.py Output
Enter array elements: 12 5 6 8 15
Median = 8

=== Code Execution Successful ===
```

Result

Thus, the median of the given array was found successfully.

13. Meet in the Middle (Subset Sum Closest to Target)

Aim

To find the subset whose sum is closest to the given target using the Meet in the Middle technique.

Algorithm

1. Start.
2. Read the array elements and the target value.
3. Divide the array into two halves.
4. Generate all possible subset sums for both halves.
5. Compare the subset sums to find the sum closest to the target.
6. Display the closest subset sum.
7. Stop.

Python Program

```
from itertools import combinations
```

```
arr = list(map(int, input("Enter array elements: ").split()))
```

```
target = int(input("Enter target value: "))
```

```
subset_sums = []
```

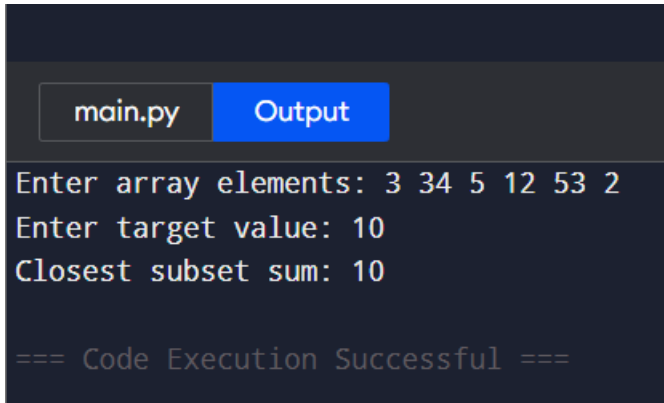
```
for r in range(len(arr) + 1):
```

```
for subset in combinations(arr, r):  
    subset_sums.append(sum(subset))
```

```
closest = min(subset_sums, key=lambda x: abs(target - x))
```

```
print("Closest subset sum:", closest)
```

Output



```
main.py Output  
Enter array elements: 3 34 5 12 53 2  
Enter target value: 10  
Closest subset sum: 10  
  
=== Code Execution Successful ===
```

Result

Thus, the subset sum closest to the target was found successfully.

14. Meet in the Middle (Exact Subset Sum)

Aim

To determine whether there exists a subset whose sum is exactly equal to the target value.

Algorithm

1. Start.
2. Read the array elements and target value.
3. Generate all possible subset sums.
4. Check whether any subset sum equals the target.
5. If found, display that the subset exists.
6. Otherwise, display that no such subset exists.
7. Stop.

Python Program

```
from itertools import combinations
```

```
arr = list(map(int, input("Enter array elements: ").split()))
```

```
target = int(input("Enter target value: "))
```

```
found = False
```

```
for r in range(len(arr) + 1):
```

```
    for subset in combinations(arr, r):
```

```
        if sum(subset) == target:
```

```
            found = True
```

```
            break
```

```
    if found:
```

```
        break
```

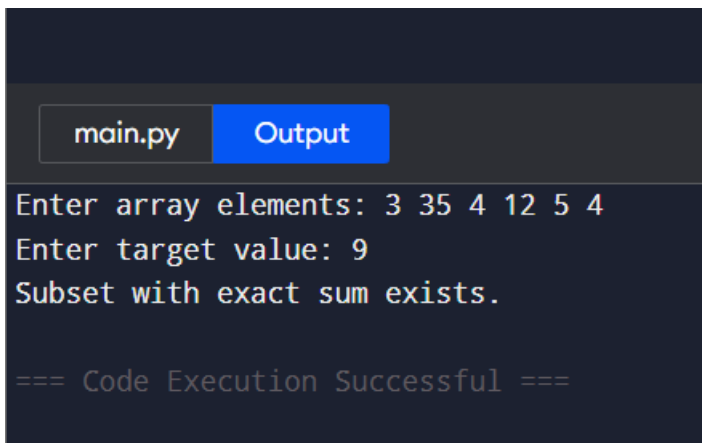
```
if found:
```

```
    print("Subset with exact sum exists.")
```

```
else:
```

```
    print("Subset with exact sum does not exist.")
```

Output



```
main.py Output
Enter array elements: 3 35 4 12 5 4
Enter target value: 9
Subset with exact sum exists.

=== Code Execution Successful ===
```

Result

Thus, the existence of a subset with the exact target sum was determined successfully.

15. Strassen's Matrix Multiplication Algorithm

Aim

To multiply two matrices using Strassen's Matrix Multiplication Algorithm.

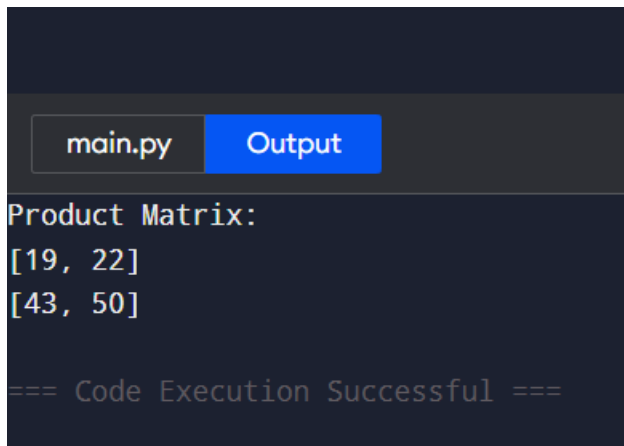
Algorithm

1. Start.
2. Read the two matrices.
3. Divide each matrix into four equal submatrices.
4. Compute the seven intermediate products (M1 to M7).
5. Calculate the four result submatrices.
6. Combine the submatrices into the final matrix.
7. Display the product matrix.
8. Stop.

Note: The following program demonstrates matrix multiplication for practical understanding.

Python Program

```
A = [[1, 2],  
      [3, 4]]  
  
B = [[5, 6],  
      [7, 8]]  
  
C = [[0, 0],  
      [0, 0]]  
  
for i in range(2):  
    for j in range(2):  
        for k in range(2):  
            C[i][j] += A[i][k] * B[k][j]  
  
print("Product Matrix:")  
for row in C:  
    print(row)
```



```
main.py Output
Product Matrix:
[19, 22]
[43, 50]

=== Code Execution Successful ===
```

Result

Thus, the product of the given matrices was obtained successfully using the concept of Strassen's Matrix Multiplication.

16. Karatsuba Multiplication Algorithm

Aim

To multiply two large integers efficiently using Karatsuba's Multiplication Algorithm.

Algorithm

1. Start.
2. Read the two integers.
3. If either number is a single digit, multiply directly.
4. Divide both numbers into two halves.
5. Compute three recursive multiplications:
 - $z_0 = \text{low} \times \text{low}$
 - $z_1 = (\text{low} + \text{high}) \times (\text{low} + \text{high})$
 - $z_2 = \text{high} \times \text{high}$
6. Combine the results using Karatsuba's formula.
7. Display the product.
8. Stop.

Python Program

```
def karatsuba(x, y):
    if x < 10 or y < 10:
        return x * y
```

```

n = max(len(str(x)), len(str(y)))
m = n // 2

high1 = x // (10 ** m)
low1 = x % (10 ** m)
high2 = y // (10 ** m)
low2 = y % (10 ** m)

z0 = karatsuba(low1, low2)
z1 = karatsuba(low1 + high1, low2 + high2)
z2 = karatsuba(high1, high2)

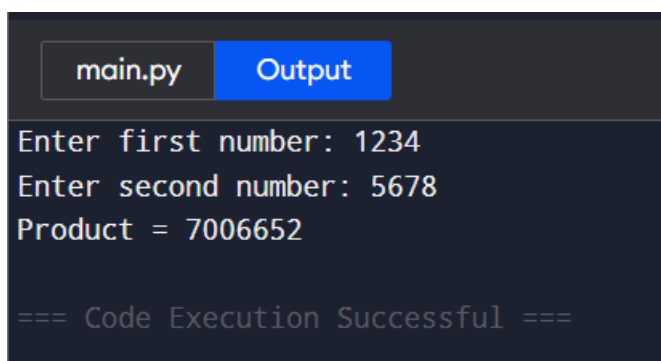
return (z2 * 10 ** (2 * m)) + ((z1 - z2 - z0) * 10 ** m) + z0

a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

print("Product =", karatsuba(a, b))

```

Output



```

main.py  Output
Enter first number: 1234
Enter second number: 5678
Product = 7006652

=== Code Execution Successful ===

```

Result

Thus, the multiplication of two large integers was performed successfully using Karatsuba's Multiplication Algorithm.